



École Hexagone

Mini TP - Déployer une API ML avec FastAPI et Docker

M1 Intelligence Artificielle

Durée : 40 minutes

Objectif pédagogique

- À la fin de ce TP, vous serez capables de :
- entraîner un modèle de régression sur un dataset classique ;
 - sauvegarder un modèle ML entraîné ;
 - exposer ce modèle via une API REST avec FastAPI ;
 - conteneuriser l'application avec Docker ;
 - tester l'API localement.

Contexte

Une agence immobilière souhaite estimer automatiquement la valeur d'un logement à partir de ses caractéristiques.

Vous disposez du dataset **California Housing**, disponible via Scikit-Learn.

L'objectif est de construire une API capable de recevoir les caractéristiques d'un logement et de retourner une prédiction de prix.

Le modèle utilisé pourra être :

- une régression linéaire ;
- une random forest ;
- ou un modèle XGBoost.

1 Partie 1 - Structure du projet

Créer un dossier nommé :

```
housing-api/
```

Dans ce dossier, créer les fichiers suivants :

```
train.py
app.py
requirements.txt
Dockerfile
```

2 Partie 2 - Entraînement du modèle

Dans le fichier `train.py`, vous devez :

1. charger le dataset California Housing ;
2. séparer les variables explicatives et la variable cible ;
3. créer un jeu d'entraînement et un jeu de test ;
4. entraîner un modèle de régression ;
5. sauvegarder le modèle entraîné dans un fichier `model.joblib`.

Le modèle doit prédire la valeur médiane d'un logement à partir des variables suivantes :

- `MedInc` : revenu médian ;
- `HouseAge` : âge moyen des logements ;
- `AveRooms` : nombre moyen de pièces ;
- `AveBedrms` : nombre moyen de chambres ;
- `Population` : population du quartier ;
- `AveOccup` : occupation moyenne ;
- `Latitude` : latitude ;
- `Longitude` : longitude.

Exécuter ensuite le script :

```
python train.py
```

À la fin de cette étape, le fichier suivant doit être généré :

```
model.joblib
```

3 Partie 3 - Création de l'API FastAPI

Dans le fichier `app.py`, vous devez créer une API avec :

- un endpoint `/health` permettant de vérifier que l'API fonctionne ;
- un endpoint `/predict` permettant de prédire la valeur d'un logement ;
- un schéma d'entrée basé sur les variables du dataset ;
- le chargement du modèle sauvegardé avec `joblib`.

L'endpoint `/predict` doit recevoir une entrée au format JSON contenant les caractéristiques du logement.

Exemple d'entrée :

```
{
  "MedInc": 8.3,
  "HouseAge": 41,
  "AveRooms": 6.9,
  "AveBedrms": 1.0,
  "Population": 322,
  "AveOccup": 2.5,
  "Latitude": 37.88,
  "Longitude": -122.23
}
```

Exemple de sortie attendue :

```
{
  "predicted_house_value": 4.85
}
```

La valeur exacte peut varier selon le modèle choisi.

4 Partie 4 - Dépendances

Dans le fichier `requirements.txt`, ajouter les dépendances nécessaires :

- FastAPI;
- Uvicorn;
- Scikit-Learn;
- Joblib;
- XGBoost, si vous choisissez ce modèle.

5 Partie 5 - Test local sans Docker

Installer les dépendances :

```
pip install -r requirements.txt
```

Entraîner le modèle :

```
python train.py
```

Lancer l'API :

```
uvicorn app:app --reload
```

Ouvrir Swagger UI :

<http://127.0.0.1:8000/docs>

Tester l'endpoint `/predict` avec l'exemple JSON fourni.

6 Partie 6 - Dockerisation

Créer un fichier `Dockerfile` permettant de :

1. partir d'une image Python légère ;
2. définir un répertoire de travail ;
3. copier les dépendances ;
4. installer les dépendances ;
5. copier le code source ;
6. exposer le port 8000 ;
7. lancer l'application FastAPI avec Uvicorn.

Construire l'image Docker :

```
docker build -t housing-api .
```

Lancer le conteneur :

```
docker run -p 8000:8000 housing-api
```

Tester ensuite l'API Dockerisée :

<http://localhost:8000/docs>

7 Partie 7 - Vérifications et debugging

Vérifier que le conteneur tourne :

```
docker ps
```

Afficher les logs :

```
docker logs <container_id>
```

Répondre brièvement aux questions suivantes :

1. Pourquoi faut-il sauvegarder le modèle entraîné ?
2. Pourquoi le modèle est-il chargé au démarrage de l'API ?
3. Pourquoi utilise-t-on 0.0.0.0 dans Docker ?
4. Quelle est la différence entre entraîner un modèle et servir un modèle ?
5. Pourquoi Docker est-il utile pour déployer une API ML ?

Bonus

Ajouter un endpoint `/predict_batch` permettant de prédire plusieurs logements en une seule requête.

L'entrée devra être une liste d'objets JSON.